

Challenge Care Plus

Documento Descritivo das Funcionalidades da Solução

Fase do Projeto:	Sprint 3 — Implementação, Integração e Iteração Final
Disciplinas:	Computational Thinking with Python & Front-End Design
Contexto:	Plataforma de engajamento em saúde, gamificação e rotinas Care Plus
Arquivo de Identificação:	INTEGRANTES.TXT (Disponível na raiz do pacote)

1. Visão Geral da Solução

A solução desenvolvida para a Sprint 3 do Challenge Care Plus consiste em uma plataforma web completa voltada ao estímulo de hábitos saudáveis por meio da gamificação (missões, trilhas de evolução e recompensas). A arquitetura é baseada no modelo cliente-servidor, utilizando um ecossistema desacoplado onde o **Python (Flask)** gerencia toda a inteligência de negócios, validações e persistência, e a interface em **HTML5, TailwindCSS e JavaScript Assíncrono** atua de forma responsiva para o usuário final.

2. Funcionalidades do Backend (Python / Flask)

O ecossistema Python foi estruturado seguindo o princípio da responsabilidade única, com divisão clara em rotas (Blueprints), serviços e utilitários. Abaixo estão descritas as principais funcionalidades implementadas:

2.1. Persistência de Dados Baseada em JSON

A persistência de dados é feita de maneira estruturada e sem a necessidade de instâncias externas de SGBD tradicionais, utilizando arquivos no formato JSON localizados no diretório `backend/data/`:

- `missions.json`: Armazena o catálogo completo de missões disponíveis, categorizadas por tipo (ex: Diária, Semanal) e ramificações de bem-estar.
- `goals.json`: Registra os objetivos de saúde e metas de evolução pessoal traçadas pelo usuário.
- `database.json` e `trails.json`: Mantêm o estado atual do usuário, níveis de experiência (XP) e progresso nas trilhas institucionais.

2.2. Operações Completas de CRUD

O backend gerencia o ciclo de vida completo dos dados através de endpoints REST, cobrindo os quatro pilares do CRUD:

Operação	Endpoint HTTP	Descrição Técnica
Create (Inserção)	POST /api/goals	Recebe novos objetivos inseridos pelo usuário no frontend, valida os campos e persiste no arquivo <code>goals.json</code> .
Read (Consulta)	GET /api/missions GET /api/mission-detail	Lista todas as missões cadastradas ou filtra dinamicamente os detalhes de uma única missão a partir do parâmetro <code>?id=X</code> recebido na URL.
Update (Alteração)	PUT /api/missions/<int:id>/complete	Altera o estado de uma missão para "concluída", aciona o motor de cálculo de recompensas e atualiza os arquivos de dados.
Delete (Exclusão)	DELETE /api/goals/<int:id>	Remove uma meta específica do arquivo <code>goals.json</code> baseando-se em seu identificador único.

2.3. Validações e Tratamento de Erros Robustos

O sistema adota tratamento preventivo e defensivo de exceções com blocos `try-except` nativos do Python, garantindo que falhas na manipulação de arquivos ou payloads malformados não derrubem a aplicação.

```
# Exemplo conceitual do tratamento aplicado em backend/utils/file_manager.py
import json

def ler_dados_json(caminho_arquivo):
    try:
        with open(caminho_arquivo, 'r', encoding='utf-8') as f:
            return json.load(f)
    except FileNotFoundError:
        # Tratamento caso o arquivo de persistência seja removido
        return []
    except json.JSONDecodeError:
        # Tratamento para evitar quebra em caso de corrupção do JSON
        return []
    except Exception as e:
        return {"error": f"Erro inesperado: {str(e)}"}
```

3. Funcionalidades do Frontend (Design e Interatividade)

A interface gráfica foi concebida sob a ótica de uma Multi-Page Application (MPA) de alta fidelidade visual, estruturada de forma limpa e sem dependências pesadas de frameworks client-side complexos.

3.1. Integração de Módulos (Missions ⇒ Detail)

A comunicação entre a tela de listagem de desafios (`missions.html`) e a tela de detalhamento (`detail.html`) é feita de forma assíncrona. Quando o usuário clica em um card gerado via JavaScript (`missions.js`), a aplicação captura o ID único e redireciona para a rota parametrizada.

O script `detail.js` extrai os parâmetros da URL em tempo de execução usando o objeto nativo `URLSearchParams` e dispara uma requisição `fetch()` ao backend do Flask localizado no `Render`, populando a árvore do DOM dinamicamente com os dados limpos em UTF-8.

3.2. Estilização com TailwindCSS e CSS Grid Layout

O layout estruturado de telas complexas como a listagem de missões e painéis de progresso foi desenvolvido utilizando o modelo avançado de **CSS Grid Layout** através do TailwindCSS. Isso permite o alinhamento perfeito de cards de conteúdo de tamanhos variados, mantendo um espaçamento harmônico e flexível.

3.3. Responsividade Completa e Elementos Interativos

A aplicação foi projetada seguindo a premissa *Mobile-First*. Utilizando os breakpoints do Tailwind (`md:`, `lg:`), a interface adapta-se perfeitamente de displays móveis de smartphones até monitores desktop de alta resolução. Elementos interativos incluem modais de confirmação de conclusão de tarefas, menus drop-down para filtragem por categoria e formulários de inserção com feedback imediato de validação.

4. Arquitetura de Deploy e Publicação

Infraestrutura em Produção

Frontend (Vercel): Publicado de forma estática com otimização de caminhos através de um arquivo de configuração `vercel.json`, eliminando erros 404 e tratando amigavelmente as rotas da MPA (`/missions`, `/detail`, `/profile`).

Backend (Render): Servidor Python/Flask ativo processando as requisições assíncronas externas de forma segura através de políticas cross-origin configuradas pelo módulo `flask-cors`.